Object-Oriented Programming

A productive approach to defining new classes is to determine what instance attributes each object should have and what class attributes each class should have. First, describe the type of each attribute and how it will be used, then try to implement the class's methods in terms of those attributes.

Q1: Smart Fridge

The `SmartFridge` class is used by smart refrigerators to track which items are in the fridge and let owners know when an item has run out.

The class internally uses a dictionary to store items, where each key is the item name and the value is the current quantity. The `add_item` method should add the given quantity of the given item and report the current quantity. You can assume that the `use_item` method will only be called on items that are already in the fridge, and it should use up the given quantity of the given item. If the quantity would fall to or below zero, it should only use *up to* the remaining quantity, and remind the owner to buy more of that item.

Finish implementing the `SmartFridge` class definition so that its `add_item` and `use_item` methods work as specified by the doctests.

> You may find Python's formatted string literals, or f-strings useful. A quick example: »> feeling = 'love' »> course = '61A!' »> f'I {feeling} {course}' 'I love 61A!'
>
> If you're curious about alternate methods of string formatting, you can also check out an older method of Python string formatting. A quick example:

```
>>> ten, twenty, thirty = 10, 'twenty', [30]
>>> '{0} plus {1} is {2}'.format(ten, twenty, thirty)
'10 plus twenty is [30]'
```

```
class SmartFridge:
    """
    >>> fridgey = SmartFridge()
    >>> fridgey.add_item('Mayo', 1)
    'I now have 1 Mayo'
    >>> fridgey.add_item('Mayo', 2)
    'I now have 3 Mayo'
    >>> fridgey.use_item('Mayo', 2.5)
    'I have 0.5 Mayo left'
    >>> fridgey.use_item('Mayo', 0.5)
    'Oh no, we need more Mayo!'
    >>> fridgey.add_item('Eggs', 12)
    'I now have 12 Eggs'
    >>> fridgey.use_item('Eggs', 15)
    'Oh no, we need more Eggs!'
    >>> fridgey.add_item('Eggs', 1)
    'I now have 1 Eggs'
    """
    def __init__(self):
        self.items = {}
    def add_item(self, item, quantity):
        if item in self.items:
            self.items[item] += quantity
        else:
            self.items[item] = quantity
        return f'I now have {self.items[item]} {item}'
    def use_item(self, item, quantity):
        self.items[item] -= min(quantity, self.items[item])
        if self.items[item] == 0:
            return f'Oh no, we need more {item}!'
        return f'I have {self.items[item]} {item} left'
```

Q2: Keyboard

Overview: A keyboard has a button for every letter of the alphabet. When a button is pressed, it outputs its letter by calling an `output` function (such as `print`). Whether that letter is uppercase or lowercase depends on how many times the *caps lock* key has been pressed.

First, implement the `Button` class, which takes a lowercase `letter` (a string) and a one-argument `output` function, such as `Button('c', print)`.

The `press` method of a `Button` calls its `output` attribute (a function) on its `letter` attribute: either uppercase if `caps_lock` has been pressed an odd number of times or lowercase otherwise. The `press` method also increments `pressed` and returns the key that was pressed. *Hint*: `'hi'.upper()` evaluates to `'HI'`.

Second, implement the `Keyboard` class. A `Keyboard` has a dictionary called `keys` containing a `Button` (with its `letter` as its key) for each letter in `LOWERCASE_LETTERS`. It also has a list of the letters `typed`, which may be a mix of uppercase and lowercase letters.

The `type` method takes a string `word` containing only lowercase letters. It invokes the `press` method of the `Button` in `keys` for each letter in `word`, which adds a letter (either lowercase or uppercase depending on `caps_lock`) to the `Keyboard`'s `typed` list. **Important:** Do not use `upper` or `letter` in your implementation of `type`; just call `press` instead.

Read the doctests and talk about:

- Why it's possible to press a button repeatedly with `.press().press().press()`.
- Why pressing a button repeatedly sometimes prints on only one line and sometimes prints multiple lines.
- Why `bored.typed` has 10 elements at the end.

Since `self.letter` is always lowercase, use `self.letter.upper()` to produce the uppercase version.

The number of times `caps_lock` has been pressed is either `self.caps_lock.pressed` or `Button.caps_lock.pressed`.

The `output` attribute is a function that can be called: `self.output(self.letter)` or `self.output(self.letter.upper())`. You do not need to return the result.

```
LOWERCASE_LETTERS = 'abcdefghijklmnopqrstuvwxyz'

class CapsLock:
    def __init__(self):
        self.pressed = 0

    def press(self):
        self.pressed += 1

class Button:
    """A button on a keyboard.

    >>> f = lambda c: print(c, end='') # The end='' argument avoids going to new line
    >>> k, e, y = Button('k', f), Button('e', f), Button('y', f)
    >>> s = e.press().press().press()
    eee
    >>> caps = Button.caps_lock
    >>> t = [x.press() for x in [k, e, y, caps, e, e, k, caps, e, y, e, caps, y, e, e]]
    keyEEKeyeYEE
    >>> u = Button('a', print).press().press().press()
    A
    A
    A
    """
    caps_lock = CapsLock()

    def __init__(self, letter, output):
        assert letter in LOWERCASE_LETTERS
        self.letter = letter
        self.output = output
        self.pressed = 0

    def press(self):
        """Call output on letter (maybe uppercased), then return the button that was
    pressed."""
        self.pressed += 1
        if self.caps_lock.pressed % 2 == 1:
            self.output(self.letter.upper())
        else:
            self.output(self.letter)
        return self
```

The keys can be created using a dictionary comprehension: `self.keys = {c: Button(c, ...) for c in LETTERS }`. The call to `Button` should take `c` and **an output function that appends to `self.typed`**, so that every time one of these buttons is pressed, it appends a letter to `self.typed`.

Call the `press` method of `self.key[w]` for each `w` in `word`. It should be the case that when you call `press`, the `Button` is already set up (in the `Keyboard.__init__` method) to output to the `typed` list of this `Keyboard`.

```
class Keyboard:
    """A keyboard.

    >>> Button.caps_lock.pressed = 0  # Reset the caps_lock key
    >>> bored = Keyboard()
    >>> bored.type('hello')
    >>> bored.typed
    ['h', 'e', 'l', 'l', 'o']
    >>> bored.keys['l'].pressed
    2

    >>> Button.caps_lock.press()
    >>> bored.type('hello')
    >>> bored.typed
    ['h', 'e', 'l', 'l', 'o', 'H', 'E', 'L', 'L', 'O']
    >>> bored.keys['l'].pressed
    4
    """
    def __init__(self):
        self.typed = []
        # END SOLUTION
        # BEGIN SOLUTION NO PROMPT ALT="# Try a dictionary comprehension!"
        self.keys = {c: Button(c, self.typed.append) for c in LOWERCASE_LETTERS}

    def type(self, word):
        """Press the button for each letter in word."""
        assert all([w in LOWERCASE_LETTERS for w in word]), 'word must be all lowercase'
        for w in word:
            self.keys[w].press()
```